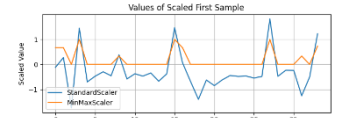


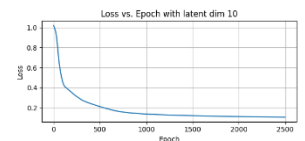
[Approach and Solution Outline]

First, I removed samples with missing values represented as the symbol "?" in dermatology dataset. After this, I applied two types of scaling: standard and minmax, resulting in X_{std} and X_{minmax} , respectively. The graph above represents the scaled values after scaling. According to the graph, while X_{std} is distributed centered zero in both negative and positive directions, minmax scaler limits X_{minmax} to the range [0,1], making it more sensitive to outliers. It's because if there is an abnormally high value in X , then minmax scaling will make almost every value other than it close to 0. Therefore, I chose standard scaling for preprocessing, and used X_{std} as raw feature. I compared k-Means clusterings plots using X_{std} and X_{minmax} in **page 4**.



```
self.encoder = nn.Sequential(
    nn.Linear(in_dim, 24),
    nn.ReLU(),
    nn.Linear(24, latent_dim)
)
self.decoder = nn.Sequential(
    nn.Linear(latent_dim, 24),
    nn.ReLU(),
    nn.Linear(24, in_dim)
)
```

From the raw feature (X_{std}), I created X_{pca} with PCA applied and X_{ae} which is an output latent feature from the encoder layers of auto-encoder I designed. X_{pca} required 22 components to retain 95% of the variance. The structure of auto-encoder is as shown on the left, with Adam optimizer, MSE loss function, learning rate = 0.001, and random seed = 42. To find out a proper latent dimension for X_{ae} which can significantly compress the data while maintaining clustering quality, I firstly chose 10 dimensions. Since the dimension of unknown dataset is 10, I thought 10 is enough to represent the overall features of the data, and also 10 is much smaller than 22 (the number of components of PCA). I'll explain later, but as a result, the clustering results remained stable, validating that 10 dimensions were sufficient. I trained the auto-encoder with the raw feature for 2500 epochs to reach a loss plateau. All models were evaluated across 3 feature spaces: raw, PCA, and AE. I chose the best feature space for each algorithm based on metrics, primarily on Silhouette scores.



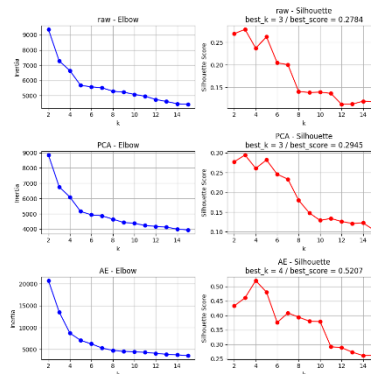
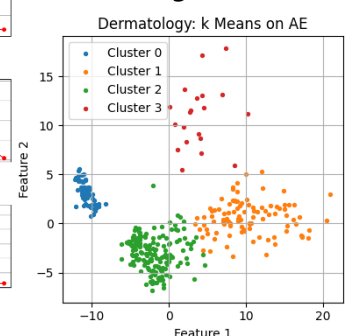
I selected 3 clustering algorithms: k-Means, Agglomerative, and DBSCAN that were covered in class. For k-Means, I used two metrics, Elbow method (inertia plot) and Silhouette score, to determine the optimal hyperparameter k . For Agglomerative, I compared dendrogram and Silhouette across three linkage methods: ward, average, and complete. For DBSCAN, I plotted heatmaps of the number of clusters, Silhouette score, and Noise Ratio for various combinations of ϵ and $\minPts$. In DBSCAN, I encountered an issue where Silhouette scores were misleading due to noise points. I diagnosed the problem as almost every Silhouette score of heatmap was negative, and addressed it by excluding noise labels (-1) when computing Silhouette scores.

In addition, I created $X_{unknown}$ which contains samples of "unknown_dataset.csv" which had already been preprocessed with PCA. I checked whether there is a missing value in the dataset, and tried to drop such samples. (Actually, there is no missing value in it.) I repeated the whole steps from plotting to inspecting which hyperparameters are adequate.

[Results and Analysis] – Answers for Analysis Questions**(a) Correct Cluster-Count Selection & Clear Plot**

	Elbow point	High score points	Optimal k
raw	k=5	k=2, 3, 5	k=5
PCA	k=5	k=3, 5	k=5
AE	k=4	K=4	k=4

<Chart 1>

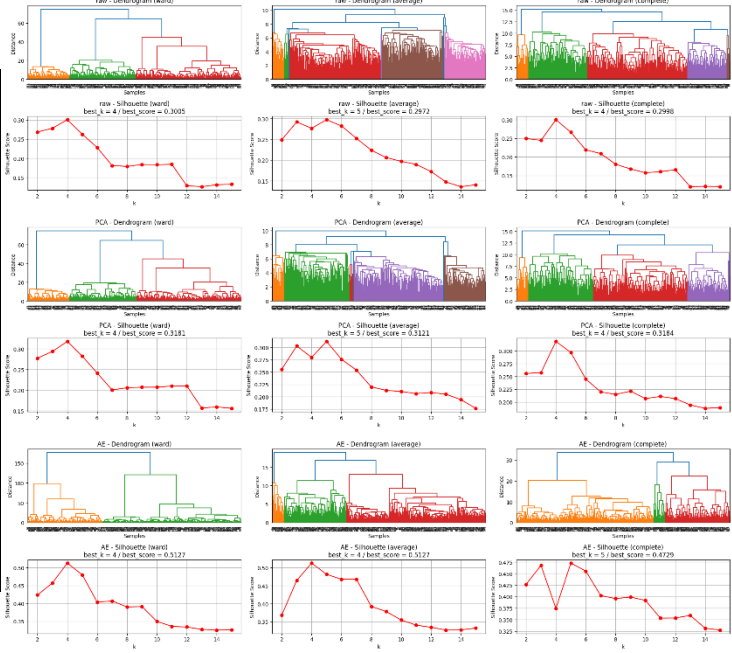
**<Figure 1>****<Figure 2>**

(i) I plotted Elbow and Silhouette curves for k ranging

from 2 to 15. From <Figure 1>, we can construct <Chart 1>. The column "High score points" means a set of k at which the points have high Silhouette scores. For raw feature, since k=5 is an elbow point and it has high score at k=5, the optimal k is 5. For PCA and AE, I applied the same strategy to determine the optimal k. <Figure 2> is the final k-Means clustering plot on X_ae with k=4. The reason I chose X_ae instead of X_std and X_pca is it has the highest score than the others.

<Chart 2>

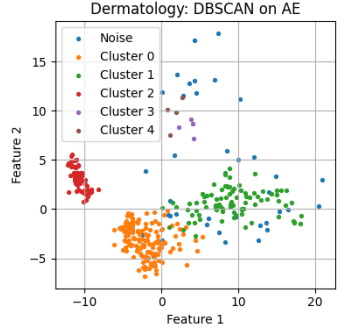
	dendrogram	High score points	Optimal k
raw (ward)	k=3, 4	k=4	k=4
raw (average)	k=5	k=3, 5	k=5
raw (complete)	k=4	k=4	k=4
PCA (ward)	k=3, 4, 5	k=4	k=4
PCA (average)	k=4, 5	k=3, 5	k=5
PCA (complete)	k=4	k=4	k=4
AE (ward)	k=2, 3, 4	k=4	k=4
AE (average)	k=3, 4	k=4	k=4
AE (complete)	k=3, 4, 5	k=3, 5	k=5



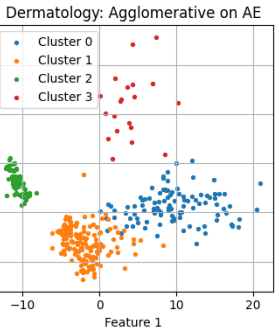
(ii) <Figure 3> is a set of dendrograms and Silhouette curves across three features and three linkages. <Chart 2> shows the proper k for cutting dendrograms and for high Silhouette scores. Among three linkages, the score tends to be high with linkage "ward" on three features. Thus, I chose "ward" linkage and plotted <Figure 4> for the final Agglomerative clustering on X_ae with k=4. The reason I chose X_ae instead of X_std and X_pca is it has the highest score than the others.

<Figure 3>

<Figure 5>

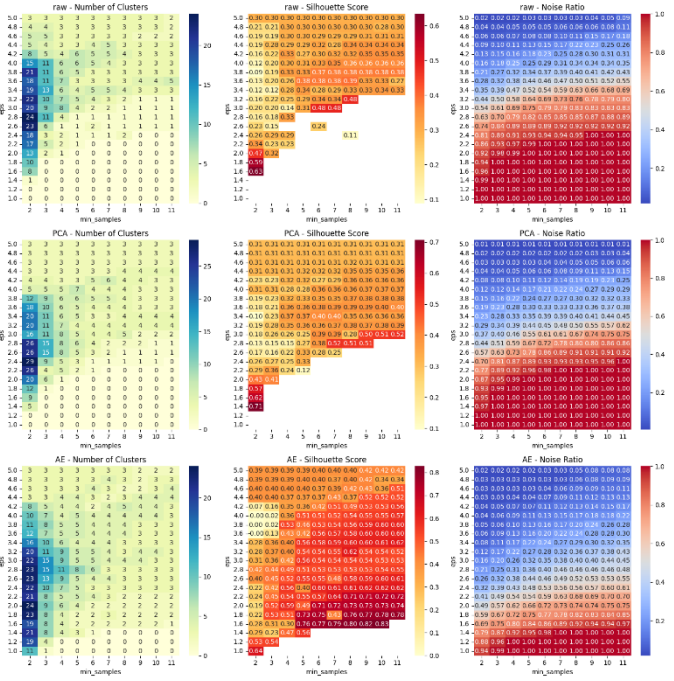


(iii) <Figure 5> is the final DBSCAN clustering plot on X_ae with eps=3.8 and minPts=4. Not only considering the number of clusters and Silhouette scores, but I also did additional analysis on noise ratio. <Figure 6> shows heatmaps for the number of clusters, Silhouette scores, and noise ratios for various combinations of eps and min_samples (minPts). Noise ratio means the rate of noise points among all sample points. I introduced this new concept, noise ratio, because when I chose eps and min_samples for the high scores with proper number of clusters and I plotted the clusters using them, it showed that almost all points were labeled as -1 (noise).



<Figure 4>

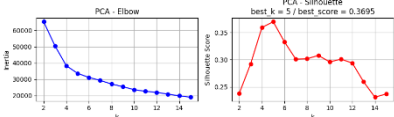
The process for choosing the optimal eps and min_samples is quite complicated. First, examine high scores points, and then check the corresponding number of clusters. If the number of clusters is 0, 1, or 2, it is too small to cluster data properly. Otherwise, if the number of clusters is close or over 10, it is too large to cluster data properly. When we find out proper combinations of eps and min_samples, then check the corresponding noise ratio. If the noise ratio is over 0.2, then it has too large amounts of noise. By considering comprehensively



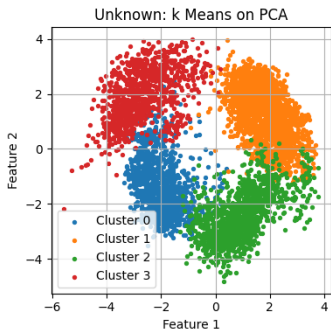
<Figure 6>

these conditions, find the optimal combination of eps and min_samples. The reason I chose X_ae instead of X_std and X_pca is it has the highest score than the others.

<Figure 7>

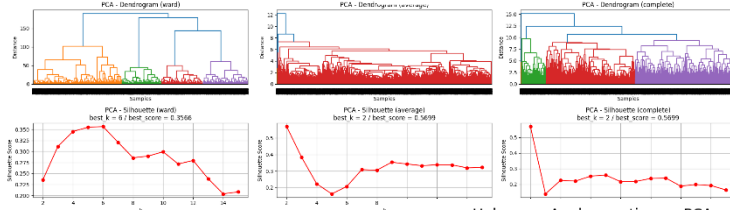


<Figure 8>



(iv) For unknown datasets, I repeated the same steps and generated three clustering plots: k-Means, Agglomerative, and DBSCAN. <Figure 7> shows Elbow and Silhouette curves on unknown datasets. According to it, k=4 is elbow point and score at k=4 is high enough, so that the optimal k is 4. I plotted the final k-Means clustering on X_unknown with k=4 in <Figure 8>.

<Figure 9>



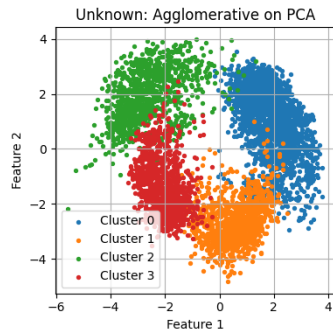
(v) <Figure 9> is a set of dendrograms and Silhouette curves on X_unknown across three linkages. <Chart 3>

	dendrogram	High score points	Optimal k
unknown (ward)	k=4	k=4, 5, 6	k=4
unknown (average)	k=1	k=2	(uncertain)
unknown (complete)	k=3	k=2	(uncertain)

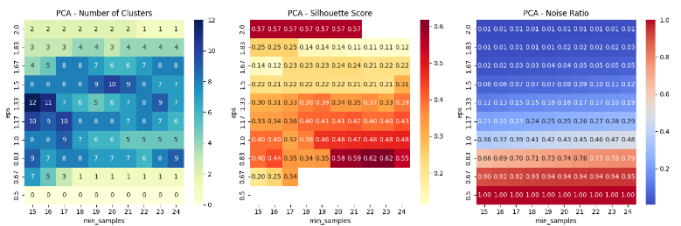
shows the proper k for cutting dendrograms and for high Silhouette scores. Among three linkages, I

<Chart 3>

can't determine the optimal k for "average" and "complete", so I plotted <Figure 10> for the final Agglomerative clustering on X_unknown with k=4 and "ward" linkage.

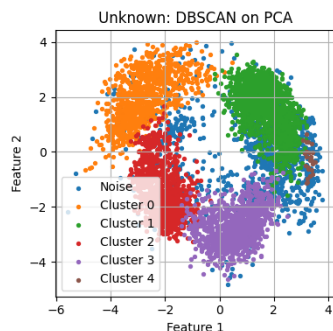


<Figure 10>



<Figure 11>

and min_samples=19 because the number of clusters at that point (5) is quite reasonable, Silhouette score (0.39) is higher than those of k-Means and Agglomerative, and noise ratio (0.16) is low. Then, I got the final DBSCAN clustering plot, <Figure 12>.



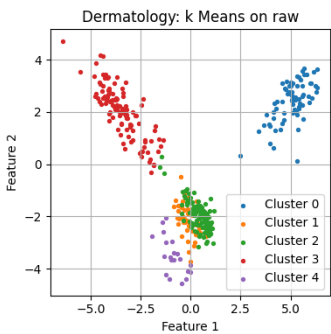
<Figure 12>

(b) Metric Agreement/Conflict Discussion

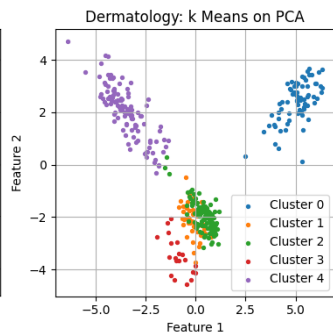
I observed both alignment and conflict between metrics. For k-Means, Elbow method and Silhouette score generally agreed well. In most cases, the elbow point also coincided with a peak or high value in the Silhouette score. This consistency made me easy to choose the optimal k. This kind of alignment can happen when intra-cluster similarity is high and inter-cluster separation is clear. In contrast, for DBSCAN, there was some trade-off between high Silhouette scores and the number of noise points. This led me to introduce a third metric, noise ratio. This kind of conflict might occur because if eps is small, then the number of noises increases, and if eps is large, then it will merge distinct clusters and lower the score.

(c) Representation Effect

I plotted k-Means clustering using raw and PCA features with k=5 which is the optimal k for them (refer to <Chart 1>). Comparing <Figure 13> and <Figure 14> with <Figure 2>, the clusters from raw and PCA look not that separated and there are some quite large overlapping regions especially between class 1 and 2 for both features. In contrast, the AE clustering shows more



<Figure 13>



<Figure 14>

compact and distinct cluster boundaries. Also, the Silhouette score from AE is much higher than those from raw and PCA features (almost twice) (refer to <Figure 1>). This is because AE can learn non-linear patterns in the data and compress them into a compact latent space, so that it shows better clustering quality. Thus, AE representation produced the clearest separation.

(d) Stability Assessment

<Figure 15> shows k-Means clusterings on X_{ae} with $k=4$ which was previously determined as optimal and with random seeds 1, 2, 3, and 4. Each run yielded different clustering labels. The clustering with seed 3 is similar to <Figure 2>, but the other clusterings highly differ from <Figure 2> and those clustering qualities are not good. This experiment shows the instability of k-Means algorithm. The cluster shapes and boundaries in k-Means depend on the initialization of centroids. This implies that the reliability of k-Means without careful initialization (k-Means++) or with just a single run might be low.

(e) Alternative Approaches or Improvements

One key weakness in the original DBSCAN pipeline is that it relies on Silhouette Score, which evaluates both intra-cluster cohesion and inter-cluster separation, making it suitable when clusters are clearly separated and compact. So, I applied a new metric, Davies-Bouldin Index. From the heatmap <Figure 16>, I chose $\epsilon=4.0$ and $\text{min_samples}=5$ since Davies-Bouldin Index of that point is a local minimum. Then, I got DBSCAN clustering plot, <Figure 17>. Compared to <Figure 5>, <Figure 17> seems to be more reasonable clustering because <Figure 5> generates two clusters in the sparse area at the top of the coordinate plane.

(appendix) Comparison k-Means clusterings using X_{std} and X_{minmax}

<Figure 18> is a k-Means clustering plot with $k=3$ on X_{minmax} , and <Figure 19> is that on X_{minmax_ae} , which is a feature space output from the auto-encoder of X_{minmax} . <Figure 20> is Elbow and Silhouette curves for X_{minmax} , X_{minmax_pca} , and X_{minmax_ae} . Compared to <Figure 13>, <Figure 14>, and <Figure 2>, the most notable difference is the scale of axes. Unlike standard scaled X_{std} , the range of axes of X_{minmax} is very small. Also, the distribution of data looks compressed and compact because minmax scaler limits all values to range [0, 1].

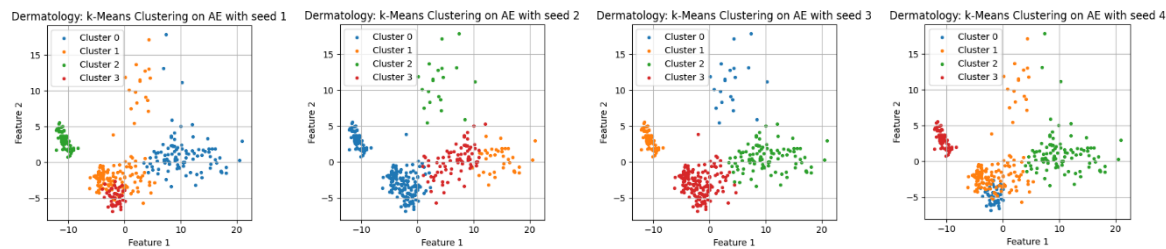
[Use of AI]

(a) Which tools or platforms have you used?

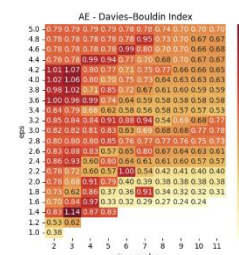
I have mainly used ChatGPT, and sometimes accepted codes suggested by autocompletion of Colab.

(b) For what tasks did you use AI and how?

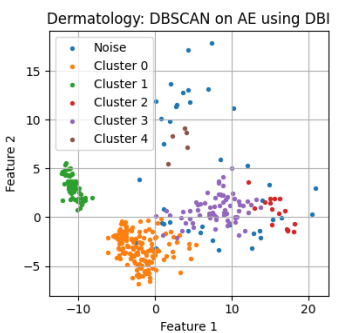
- Understanding Concepts: I asked for explanations about the meaning of inertia of



<Figure 15>

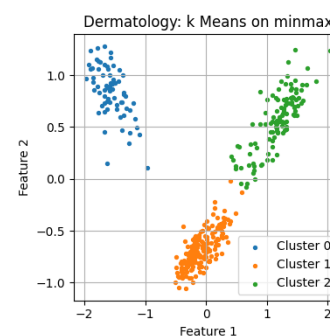


<Figure 16>

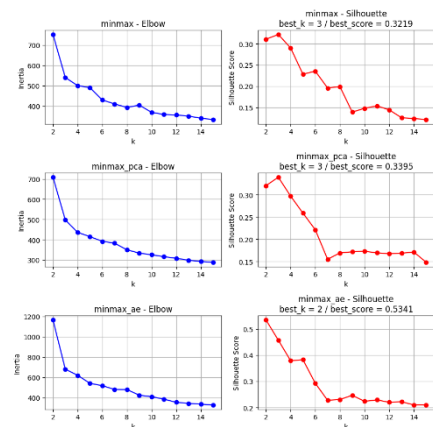
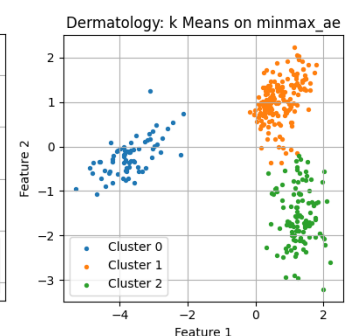


<Figure 17>

<Figure 18>



<Figure 19>



<Figure 20>

k-Means, linkage methods of Agglomerative clustering (ward, average, and complete), and heatmap, how PCA chooses the number of components and compresses the data, and the definitions (equations) of Silhouette score and Davies-Bouldin Index. I also got information about the internal implementation of methods or functions such as `np.linalg.norm`, `np.newaxis`, `np.allclose`, `np.where`, and `zip`.

- Code Suggestions: I asked ChatGPT to help me write codes for many parts such as implementation of k-Means algorithm in task 1, fixing all seeds for complete reproducibility, plotting graphs, pivoting results and displaying heatmaps in DBSCAN, and 2D scatter plots with color labels. For loading datasets, I asked ChatGPT how to fetch or install from `ucimlrepo`, and how to cache locally "dermatology_dataset.csv". I reviews and modified those generated codes, including adding titles and labels, adjusting figure sizes, and printing shapes of variables or the number of elements.

- Debugging: In implementing for plotting heatmaps in DBSCAN, I encountered an error that generated figures were just blank. So, I asked ChatGPT for the probable cause of it and how to fix it.

(c) How you integrated the AI's suggestions into your work.

Since ChatGPT just suggested codes for plotting individual figure, which is hard to compare and inspect, I modified the internal structures of each plotting function to control the overall figure size and put together several subplots in a single figure. For example, in the function 'elbow_silhouette_plot', I added some variables, and changed parameters and internal structure to integrate plotting codes. I set the types of input parameters 'features' and 'names' as lists of features and name of them, respectively. And in the function, I prepared for plotting several subplots with appropriate sizes, then I ran 'for loop' on each corresponding pair of features and names to plot each subplot. Also, I separately processed the case where the number of elements of 'features' or 'names' is one to prevent errors.

```
# Features from Dermatology dataset
features_derm = [X_std, X_pca, X_ae]
names_derm = ["raw", "PCA", "AE"]

#-----4. Helper: k Means Elbow & Silhouette-----
def elbow_silhouette_plot(features, names, k_range=range(2, 16), random_state=42):
    fig, axes = plt.subplots(len(names), 2, figsize=(9, 3*len(names)))

    for i, (X, name) in enumerate(zip(features, names)):
        inertia, sil = [], []

        # TODO: select best k by inspecting the plots
        if len(names) == 1:
            axes = [axes]
            ax0, ax1 = axes[i]

        # Elbow Plot
        ax0.plot(k_range, inertia, 'bo-')
        ax0.set_title(f'{name} - Elbow')
        ax0.set_xlabel("k")
        ax0.set_ylabel("Inertia")
        ax0.grid(True)

        # Silhouette Plot
        best_k = np.argmax(sil) + k_range.start
        best_score = np.max(sil)
        ax1.plot(k_range, sil, 'ro-')
        ax1.set_title(f'{name} - Silhouette')
        ax1.set_xlabel("k")
        ax1.set_ylabel("Silhouette Score")
        ax1.grid(True)
```

(d) Any instances where AI was wrong or not directly helpful.

When I asked ChatGPT how to fix all seeds for reproducibility, it suggested me to define a function 'set_seed' and run it once at the beginning of whole code. However, the output of latent feature space `X_ae` from auto-encoder was different for every run, indicating that fixing seeds does not work properly. So, I tried to run 'set_seed' again inside the function for training the auto-encoder, then reproducibility began to be well maintained.

Another example is that ChatGPT suggested computing Silhouette score and plotting heatmap for it in DBSCAN. I checked the result, but almost every score was negative, which means it shows terrible clustering quality. I realized that this problem must occur due to noise, so I added a masking part in the code in order not to consider cases when the label is -1 (noise).

(e) Citation of AI content.

In task 1, the code for internal implementation of k-Means algorithm is significantly generated by ChatGPT. Before writing the final implementation, I asked for what is the meaning, internal computation, or output of `np.linalg.norm`, `np.newaxis`, `np.allclose`, and `np.where`. Also, in task 2, I got a lot of aids by ChatGPT in all implementing visualization parts, especially for plotting heatmaps in DBSCAN algorithm.